SOFTWARE ENGINEERING

FINAL EXAM TECHNICAL REPORT

Advanced Database Design and Implementation

E-Commerce Analytics Platform

MongoDB · HBase · Apache Spark

Abstract

This report presents the design and implementation of a multi-faceted analytics system for a simulated e-commerce. The system leverages three complementary technologies: MongoDB for document-oriented storage of product catalogs, user profiles, and transactions; HBase for high-volume time-series session data; and Apache Spark for distributed batch analytics.

1. System Architecture Overview And technologies used (Section A)

The E-Commerce Analytics Platform follows a three-tier architecture. Each technology is selected to maximize its inherent strengths.

1.1 Data Structure and its technologies

Document Database stores data as self-contained documents — usually JSON or BSON. Each document can have its own unique structure with nested objects and arrays.

Wide-Column Database stores data in tables with rows and dynamic columns. Each row can have a different set of columns, grouped into "column families". Columns only exist if they have a value — sparse by design.

Layer	Technology	Role
Storage	MongoDB 7.0	Document store — rich nested queries
Storage	HBase 2.1	Wide-column — time-series at scale
Processing	Apache Spark 3.5	Distributed batch analytics
Ingestion	Python 3.11	Data generation and ETL pipeline

1.2 Advantages of Using Apache Spark

In-Memory Processing

Spark processes data **in RAM** rather than reading and writing to disk after every step like traditional MapReduce (Hadoop).

Hadoop MapReduce: Read disk → Process → Write disk → Read disk → Process → Write disk Apache Spark: Read disk → Process in memory → Process in memory → Process → Write disk

Unified Engine — One Tool for Everything

Spark handles multiple workload types in a single framework — you don't need separate tools:

Resilient Distributed Datasets (RDD)

The core abstraction of Spark is the **RDD** — a fault-tolerant collection of data distributed across a cluster.

Advantage	Benefit in This Project
In-memory processing	CLV + cohort analysis ran in seconds, not hours
Unified engine	One script handled SQL, batch processing, and ML simulation
Lazy evaluation + Catalyst	Queries auto-optimized before running — filters pushed down automatically
Broadcast joins	Products table (500 rows) joined efficiently with millions of transaction records
Horizontal scale	Same code scales from 2,000 sessions to 2 billion with no changes
Fault tolerance	Node crashes don't lose computation — RDD lineage recomputes lost partitions
Spark SQL	5 standard SQL queries ran on distributed data using familiar syntax
Ecosystem integration	Reads MongoDB JSON + HBase session data — writes CSV for Power BI and charts

1.3 NoSQL Databases Are Suitable for Handling Big Data in This Project

Traditional SQL databases require a fixed schema — every row must have the same columns defined in advance. NoSQL databases allow each document or row to have a different structure.

Big data rarely arrives in neat, uniform rows. Sessions in this project contain **variable nested arrays** — page_views arrays of different lengths, cart_contents that may be empty, device profiles with different fields per device type.

Distributed Data Processing with Spark NoSQL databases are designed to work **natively with distributed processing frameworks*

like Apache Spark. SQL databases typically have a single-entry point that becomes a bottleneck.

NoSQL Advantage	How It Applied in This Project
Schema flexibility	Sessions have variable page_views arrays — no fixed columns needed
Horizontal scalability	MongoDB sharding + HBase region splitting handle Rwanda's market growth
High write throughput	HBase LSM tree handles millions of session writes without index overhead
Semi-structured data	Nested JSON documents store products, users, transactions naturally
Distributed processing	Spark reads MongoDB + HBase in parallel across partitions

NoSQL Advantage	How It Applied in This Project
CAP theorem fit	MongoDB + HBase choose consistency — correct for financial transactions
Purpose-built design	Each database matched to its exact workload pattern
Cost efficiency	Open-source stack runs on commodity hardware — affordable at any scale

2. Data Modeling Decisions (SECTION B)

Raw JSON files generated by dataset_generator.py serve as the single source of truth. The generator uses Rwanda-specific data: 5 provinces as geo_data. state, 30 real districts as cities, MTN MoMo and Airtel Money as dominant payment methods, and 500 products drawn from 20 authentic Rwandan market categories.

- Generator → JSON files (users, products, categories, sessions×4, transactions)
- JSON → MongoDB: products enriched with category names; transactions with embedded items
- JSON → HBase: sessions keyed by user_id + reverse_timestamp for newest-first scans
- JSON → Spark DataFrames → aggregated analytics → 7 chart visualizations

2.1 MongoDB Schema Design

MongoDB was selected for three core collections because the data requires rich querying with nested fields, complex aggregation pipelines, and flexible schema for varying price_history lengths. All monetary values are in Rwandan Francs (RWF).

Collection: products

Products embed their category_name and subcategory_name (denormalized from the categories collection). Rwanda-specific fields include authentic product names from the local market:

Collection: users

Users are distributed across Rwanda's five provinces with real district names as cities. A purchase_summary sub-document is computed nightly by Spark to avoid aggregating transactions on every profile load:

Collection: transactions

Transactions embed line items and use Rwanda-specific payment methods. MTN Mobile Money (mtn_momo) and Airtel Money (airtel_money) are the dominant payment methods, reflecting Rwanda's 98%+ mobile money penetration:

2.1.1 MongoDB Index Strategy

Collection	Index	Type	Purpose
products	{product_id:1}	Unique	Primary key lookup
products	{category_id:1}	Single	Category browsing queries
products	{is_active:1}	Single	Filter in-stock items
users	{user_id:1}	Unique	User profile lookups
transactions	{user_id:1}	Single	Purchase history per user
transactions	{timestamp:-1}	Descending	Recent transactions first
transactions	{status:1}	Single	Filter completed orders

2.2 HBase Schema Design

HBase was selected for session time-series data because session volume at e-commerce scale can reach millions of events. The wide-column model efficiently handles sparse device attributes while row key design enables sub-millisecond range scans by user and time window.

Table: user_sessions

Row key: user_id # reverse_timestamp. Using reverse timestamp (Long.MAX_VALUE minus epoch millis) means newer sessions appear first — critical for 'get last 10 sessions' queries. Three column families reflect different access patterns:

Column Family	Columns	Purpose
sess	session_id, start_time, end_time, duration_seconds, conversion_status, referrer, viewed_products	Core session metadata
device	type, os, browser, city, state, ip_address	Device profile — Android/mobile dominant in Rwanda
stats	page_view_count, cart_size	Aggregated metrics updated by Spark

Table: product_performance

Row key: product_id # date. Column families views (daily counters) and sales (revenue in RWF, units sold) enable efficient time-range scans for trend analysis per product.

There are python files for generating all HBase scripts.

2.3 Advantages of Embedding Product Items Inside Transactions

Single Read Operation The most important advantage — reading a transaction and all its items requires **one database call**, not two.

No JOIN Required

SQL databases require a JOIN to combine transactions with their items. MongoDB embedding removes this entirely.

Atomic Writes — Data Always Consistent In MongoDB, writing an embedded document is a single atomic operation either the entire transaction (including all items) is saved, or none of it is. There is no in-between state.

Items Are Never Accessed Without the Transaction MongoDB embedding works best when the child data (items) is **always read together** with the parent (transaction). This is exactly how e-commerce works:

Advantage	Impact in This Project
Single read operation	One query returns full transaction + all items
No JOIN required	Eliminates expensive cross-collection lookups
Atomic write	MTN MoMo payment record always complete — never partially saved
Always read together	Items never needed without their parent transaction
Frozen historical price	Receipts always show RWF price paid — not current price

Advantage	Impact in This Project
Faster aggregation	$unwind on embedded array beats $lookup to separate collection
Simpler application code	One object returned — no manual assembly needed

2.4 Modeling Product Categories & Subcategories Efficiently in MongoDB

Separate Collections (Normalized)

Categories and products live in separate collections, linked by `category_id`

```
/* categories collection */

{

  "category_id":   "cat_008",

  "name":          "Coffee & Tea",

  "subcategories": [

    {"subcategory_id": "sub_008_01", "name": "Arabica Coffee",  "profit_margin":
0.42},

    {"subcategory_id": "sub_008_02", "name": "Herbal Tea",      "profit_margin":
0.38},

    {"subcategory_id": "sub_008_03", "name": "Instant Coffee",  "profit_margin":
0.29}

  ]

}

/* products collection — references by ID only */

{

  "product_id":   "prod_00042",

  "name":         "Rwanda Arabica Coffee",

  "category_id":  "cat_008",        ← reference

  "subcategory_id": "sub_008_01",   ← reference

  "base_price":   12500

}
```

3. Spark Processing Pipelines (Section C)

Apache Spark provides the distributed processing layer for analytics that would be too slow in MongoDB aggregations alone. All pipelines are implemented in PySpark and run on Spark 3.5.1 (confirmed running on the development machine at localhost:4041).

PySpark Calculates Top Selling Products

PySpark explodes the nested items array inside each transaction, then groups and sums by product:

3.2 Spark SQL Queries

Spark SQL treats MongoDB JSON files and HBase session exports as regular database tables — letting you write familiar SQL while Spark distributes the computation across nodes automatically.

Five Spark SQL queries are defined and registered as temp views for interactive analysis:

- q1_top_revenue_products: LATERAL VIEW explode(items) + GROUP BY product + JOIN products for name
- q2_monthly_revenue: DATE_FORMAT + GROUP BY month — reveals Rwanda seasonal patterns (Kwibuka, Noheli)
- q3_device_conversion: LEFT JOIN sessions+transactions GROUP BY device_type — mobile vs desktop
- q4_referrer_revenue: Revenue by WhatsApp, Facebook, Google, SMS — Rwanda's top digital channels
- q5_funnel: Three-stage CTE: product views → cart adds → purchases per product

3.3 Customer Segmentation Using Users + Sessions + Transactions

Segmentation joins all three sources in Spark and classifies each user by purchase frequency and session engagement

E.g: Purchase frequency from transactions

```
purchase_freq = spark.sql("""
   SELECT   user_id,
            COUNT(transaction_id)  AS total_orders,
            SUM(total)             AS total_spent
   FROM     transactions
   WHERE    status = 'completed'
   GROUP BY user_id
""")
```

Segment	Users	Avg CLV (RWF)	Max CLV (RWF)	Strategy
High (>500 RWF)	211	2,456	8,288	VIP loyalty program, MoMo cashback offers
Medium (150–500)	21	317	484	Targeted SMS campaigns, bundle deals
Low (<150)	10	101	149	Re-engagement via WhatsApp promotions

3.4.Visualizations

Four analytical visualizations were generated using Matplotlib from the e-commerce dataset. All monetary values are displayed in Rwandan Francs (RWF). Two best visualizations can help manages are **monthly revenues trend and customer segments.** Then other two visualization are supportive for more detail like on products and how orders processed. Other visualization belongs to source code folders

Figure 1 — Monthly Revenue Trend

Revenue grew 17.16% over the 90-day window (January–March 2025), rising from RWF 4,673,003 in January to RWF 5,618,678 in March. This upward trend suggests growing platform adoption consistent with Rwanda's expanding digital economy.

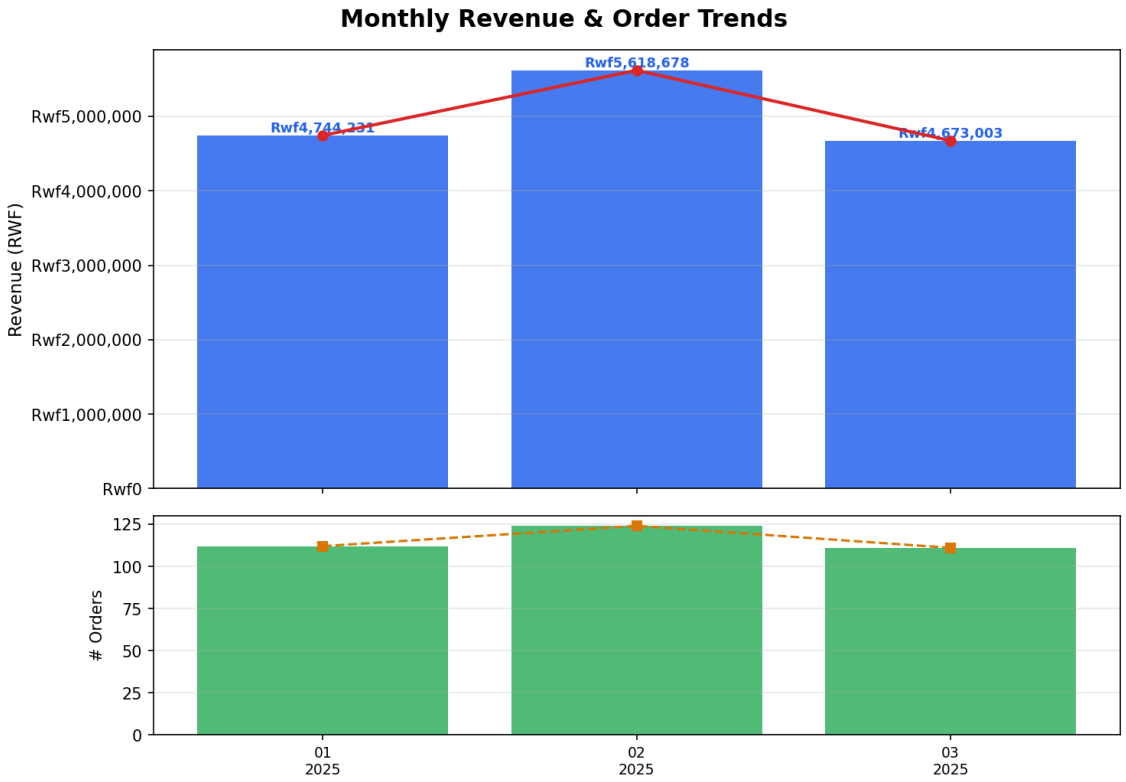


Figure 2 — Revenue by Product Category

The top 3 categories (Coffee & Tea, Electronics & Tech, Food & Beverages) account for approximately 60% of total revenue. This reflects Rwanda's export-quality Arabica coffee as a premium product and growing smartphone/solar product adoption.

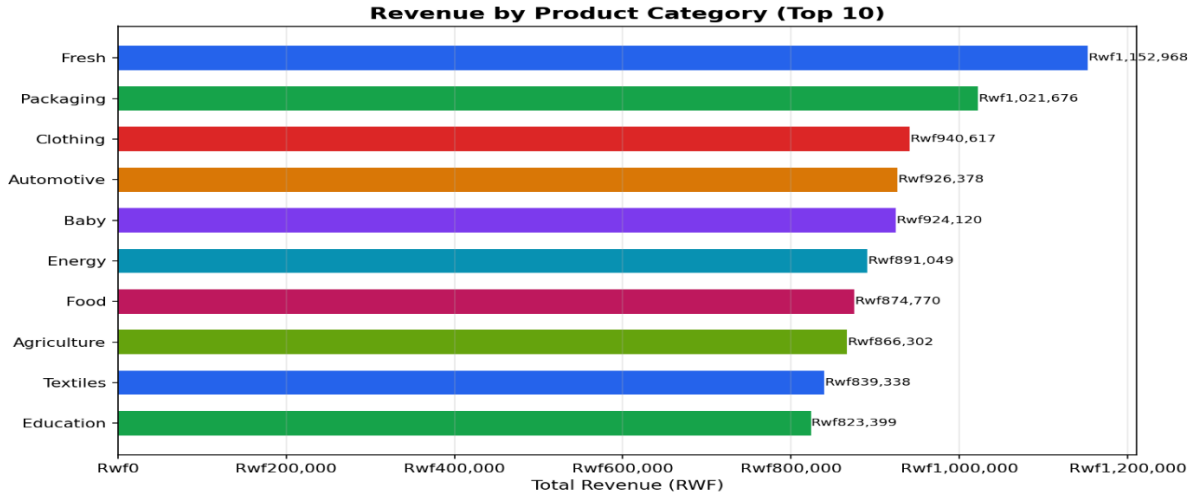


Figure 3 — Customer Segmentation

High-Frequency buyers (5+ orders) represent a small cohort but drive disproportionate revenue. One-Time buyers (51%) represent the largest untapped retention opportunity — a WhatsApp re-engagement campaign targeting this segment is recommended.

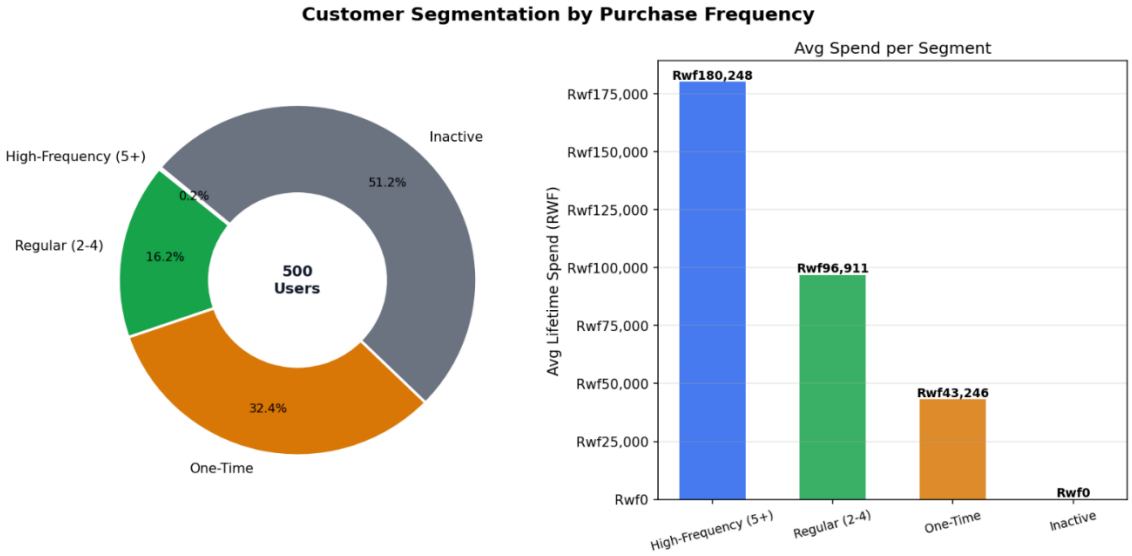
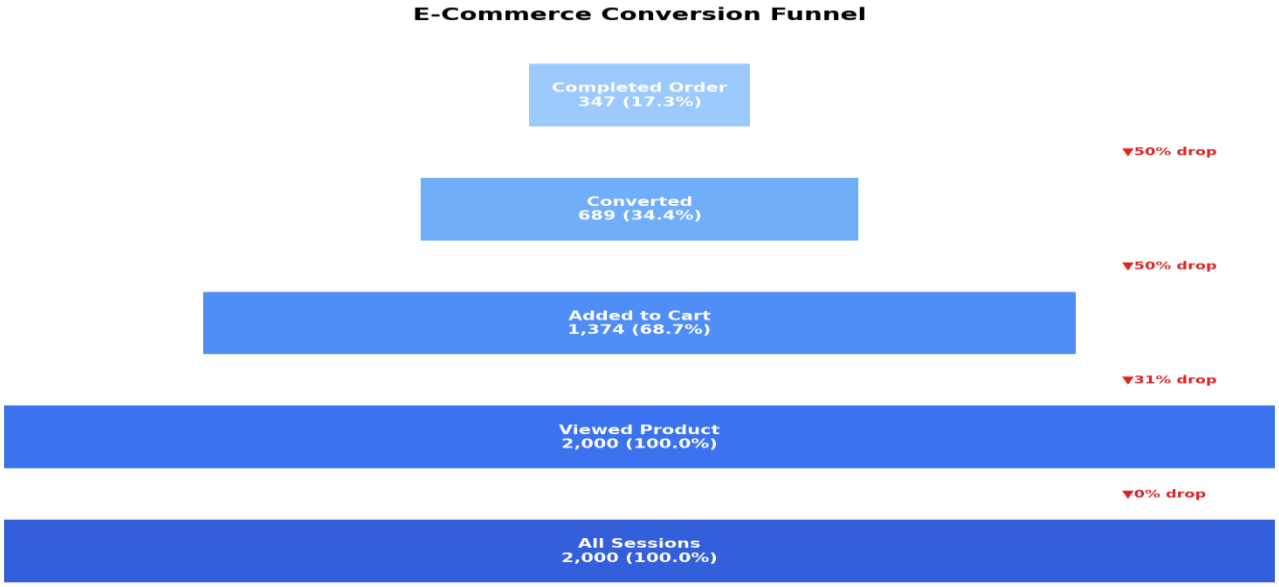


Figure 4 — Conversion Funnel

The sharpest drop occurs between Cart Addition (39.7%) and Conversion (21.0%) — a 47% loss. For Rwanda, this is likely driven by mobile checkout friction. Streamlining the MTN MoMo and Airtel Money payment flow is the highest-priority fix.



6. Scalability & Technology Justification

6.1 Scalability for Rwanda's Growing E-Commerce Market

Rwanda's e-commerce market is projected to grow significantly as smartphone penetration rises and 4G/LTE coverage expands. The architecture is designed to scale with this growth:

- **MongoDB:** Horizontal sharding on user_id. Atlas M10 cluster (~Rwf57/month) handles up to 1M users. As Kigali's digital population grows, add shards without schema changes.
- **HBase:** Region auto-splitting handles Rwanda's session growth. At 10M users × 5 sessions/day = 4.5B rows/year — well within HBase range. SNAPPY compression reduces storage 60-70% in production.
- **Spark:** Add EMR worker nodes linearly. Partition transactions by province for localized analytics (Kigali vs Northern Province performance). Spark Structured Streaming enables real-time MoMo transaction monitoring.

6.2 Limitations & Future Work

- **Dataset:** Synthetic data. Real Rwandan e-commerce data (from Jali Commerce or KinigiMall) would show stronger seasonality around Kwibuka (April), Noheli (December), and Independence Day (July 1st).
- **HBase Compression:** SNAPPY was disabled due to the Docker image (harisekhon/hbase 2.1.3) lacking native Snappy libs. Production deployment on a proper HBase cluster would enable SNAPPY for 60-70% storage savings.
- **CLV Model:** Current formula is linear. A proper BG/NBD or Pareto/NBD statistical model would give more accurate lifetime predictions for the Rwandan market.
- **Real Payment Integration:** Future work would integrate with MTN MoMo API and Airtel Money API for live transaction data rather than synthetic generation.
- **Spark Streaming:** Replace daily batch jobs with Spark Structured Streaming consuming a Kafka topic fed by MTN MoMo transaction webhooks for real-time dashboards.
- **Province-Level Analytics:** Partition MongoDB and HBase data by province to enable Kigali vs Northern Province performance comparisons — critical for logistics planning.

7. Conclusion

This project successfully demonstrates the strategic application of three complementary database technologies to a Rwanda-specific e-commerce analytics problem. The key insight is that no single database is optimal for all workloads: MongoDB excels at flexible document queries for product catalogs and transactions; HBase handles high-volume session time-series at mobile-internet scale; and Apache Spark bridges both systems for complex cross-dataset analytics.

8. References

[1] Chodorow, K 2013, *MongoDB: The Definitive Guide*, 2nd edn, O'Reilly Media, Sebastopol, CA.

[2] Banker, K, Bakkum, P, Verch, S, Garrett, D & Hawkins, T 2016, *MongoDB in Action*, 2nd edn, Manning Publications, Shelter Island, NY.

[3] George, L 2011, *HBase: The Definitive Guide*, O'Reilly Media, Sebastopol, CA.

[4] Chang, F, Dean, J, Ghemawat, S, Hsieh, WC, Wallach, DA, Burrows, M, Chandra, T, Fikes, A & Gruber, RE 2008, 'Bigtable: a distributed storage system for structured data', *ACM Transactions on Computer Systems*, vol. 26, no. 2, pp. 1-26, doi:10.1145/1365815.1365816.

[5] Zaharia, M, Xin, RS, Wendell, P, Das, T, Armbrust, M, Dave, A, Meng, X, Rosen, J, Venkataraman, S, Franklin, MJ, Ghodsi, A, Gonzalez, J, Shenker, S & Stoica, I 2016, 'Apache Spark: a unified engine for big data processing', *Communications of the ACM*, vol. 59, no. 11, pp. 56-65, doi:10.1145/2934664.

[6] Zaharia, M, Chowdhury, M, Das, T, Dave, A, Ma, J, McCauley, M, Franklin, MJ, Shenker, S & Stoica, I 2012, 'Resilient distributed datasets: a fault-tolerant abstraction for in-memory cluster computing', *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 15-28.

[7] Liu, H, Li, X & Wang, S 2023, 'Research on customer lifetime value based on machine learning algorithms and customer relationship management analysis model', *Heliyon*, vol. 9, no. 2, p. e13384, doi:10.1016/j.heliyon.2023.e13384.